University of Central Missouri
Department of Computer Science & Cybersecurity

CS5760 Natural Language Processing
Spring 2026

Homework 1

Student name: M. Siddartha Reddy
700774070

Submission Requirements:

- Once finished your assignment push your source code to your repo (GitHub) and explain the work through the ReadMe file properly. Make sure you add your student info in the ReadMe file.
- Submit your GitHub link on the Bright Space.
- Comment your code appropriately ***IMPORTANT.***
- Any submission after provided deadline is considered as a late submission.

Q1. Regex

Task: Write a regex to find

1. **U.S. ZIP codes** (disjunction + token boundaries)

   Match `12345` or `12345-6789` or `12345 6789` (hyphen or space allowed for the +4 part). Make sure you only match whole tokens (not inside longer strings).

2. **Negation in disjunction** (word start rules)

   Find all words that do not start with a capital letter. Words may include internal apostrophes/hyphens like `don't`, `state-of-the-art`.

3. **Convenient aliases** (numbers, a bit richer)

   Extract all numbers that may have:

   a. optional sign (+/-),

   b. optional thousands separators (commas),

   c. optional decimal part,

   d. optional scientific notation (e.g., `1.23e-4`).

4. **More disjunction** (spelling variants)

   Match any spelling of "email": `email`, `e-mail`, or `e mail`. Accept either a space or a hyphen (including en-dash –) between `e` and `mail`, and be case-insensitive.

5. **Wildcards, optionality, repetition** (with punctuation)

   Match the interjection `go`, `goo`, `gooo`, … (one or more `o`), as a word, and allow an optional trailing punctuation mark `! . , ?` (e.g., `gooo!`).

6. **Anchors** (line/sentence end with quotes)

   Match lines that end with a question mark possibly followed only by closing quotes/ brackets like `")"']` and spaces.

Q2. Manual BPE on a toy corpus

2.1 Using the same corpus from class:

```
low low low low low lowest lowest newer newer newer newer newer newer wider
wider wider new new
```

1. Add the end-of-word marker _ and write the *initial vocabulary* (characters + _).
2. Compute bigram counts and perform the first three merges by hand:

    - Step 1: most frequent pair → merge → updated corpus snippet (show at least 2 lines).
    - Step 2: repeat.
    - Step 3: repeat.
3. After each merge, list the new token and the updated vocabulary

2.2 — Code a mini-BPE learner

1. Use the classroom code above (or your own) to learn BPE merges for the toy corpus.
    - Print the top pair at each step and the evolving vocabulary size.
2. Segment the words: `new`, `newer`, `lowest`, `widest`, and one word you invent (e.g., `newestest`).
    - Include the subword sequence produced (tokens with _ where applicable).
3. In 5–6 sentences, explain:
    - How subword tokens solved the OOV (out-of-vocabulary) problem.
    - One example where subwords align with a meaningful morpheme (e.g., `er_` as English agent/comparative suffix).

2.3 — *Your language* (or English if you prefer)

Pick one short paragraph (4–6 sentences) in *your own language* (or English if that's simpler).

1. Train BPE on that paragraph (or a small file of your choice).
    - Use end-of-word _.
    - Learn at least 30 merges (adjust if the text is very small).
2. Show the five most frequent merges and the resulting five longest subword tokens.
3. Segment 5 different words from the paragraph:
    - Include one rare word and one derived/inflected form.
4. Brief reflection (5–8 sentences):
    - What kinds of subwords were learned (prefixes, suffixes, stems, whole words)?
    - Two concrete pros/cons of subword tokenization for your language.

Q3. Bayes Rule Applied to Text (based on slide: Bayes' Rule for documents)
The PPT shows that classification is based on:

$$c_{MAP} = \arg\max_{c \in C} P(c)\, P(d \mid c)$$

Tasks:

1. Explain in your own words what each term means: P(c), P(d|c) and P(c|d).

P(c) — Prior probability of the class

This is how likely a class is **before** looking at the document.
Example: if 70% of training emails are "spam," then P(spam)≈ 0.7

P(d|c) — Likelihood of the document given the class

This is how likely it is to see this document's words **assuming** the document truly belongs to class c. In Naive Bayes, we compute this from the word probabilities learned for that class (often as a product/sum of log-probabilities).

P(c|d) — Posterior probability of the class given the document

This is what we actually want for classification: the probability the document belongs to class c **after** reading the document.
Bayes rule connects them:

P(c|d)=P(c)P(d|c) / P(d)

2. Why can the denominator P(d) be ignored when comparing classes?

Because P(d) is the same number for every class when the document d is fixed.

When choosing the best class:

argcmax P(c|d)=argcmax P(c)P(d|c)/ P(d)

Since dividing by the same constant P(d) does **not** change which class is largest:

argcmax P(c)P(d|c) / P(d) =argcmax P(c)P(d|c)

So we ignore P(d) for classification (we only care which class wins, not the exact normalized probability).

Q4. Add-1 Smoothing (based on slide: Worked Sentiment Example)

In the worked example, priors are: P(−)=3/5, P(+)=2/5. Vocabulary size = 20.

Tasks:

1. For the negative class, the total token count is 14. Compute the denominator for likelihood estimation using add-1 smoothing.
2. Compute P(predictable|−) if the word “predictable” occurs 2 times in the negative documents.

Compute P(fun|−) if “fun” never appeared in any negative documents.

Using **add-1 (Laplace) smoothing** for a multinomial NB likelihood:

P(w | -) = {c(w,-)+1}/{N_+ |V|}

Given:

- total tokens in negative class (N- = 14)
- vocabulary size (|V| = 20)

1) Denominator (negative class)

N- + |V| = 14 + 20 = 34

2) P(predictable | -) where c(predictable,-)=2

P(predictable | -) = 2+1/34 = 3/34 = approx 0.0882

3) P(fun | -)) where c(fun,-)=0

P(fun | -) = 0+1/34 = 1/34 = approx 0.0294

Q5. Programming Question:

1. **Tokenize a paragraph**

   Take a short paragraph (3–4 sentences) in your language (e.g., from news, a story, or social media).

Paragraph (3–4 sentences)

I can’t believe it’s already February! Kishore’s team shipped a state-of-the-art demo, but users didn’t read the FAQ—so they asked, “Why’s it broken?” Yesterday, we re-tested v2.0, fixed 3 bugs, and emailed support@example.com.

- Do naïve space-based tokenization.

1. I
2. can’t
3. believe
4. it’s
5. already
6. February!
7. Kishore’s
8. team
9. shipped
10. a
11. state-of-the-art
12. demo,
13. but
14. users
15. didn’t
16. read
17. the
18. FAQ—so
19. they

20. asked,
21. “Why’s
22. it
23. broken?”
24. Yesterday,
25. we
26. re-tested
27. v2.0,
28. fixed
29. 3
30. bugs,
31. and
32. emailed
33. support@example.com.

- **Manually correct the tokens by handling punctuation, suffixes, and clitics. Submit both versions and highlight differences.**

Rules I applied:

- **Separate punctuation**: ! , — “ ” ? . become their own tokens when attached.
- **Split clitics** (common in NLP):
  - can’t → can + n’t
  - it’s → it + ’s
  - didn’t → did + n’t
  - possessive Kishore’s → Kishore + ’s
  - Why’s → Why + ’s
- **Split a few suffixes** (simple manual morphology): -ed, plural -s
- Keep email as one token; keep version v2.0 as one token.

1. I

2. can

3. n’t

4. believe

5. it

6. ’s

7. already

8. February

9. !

10. Kishore

11. ’s

12. team

13. ship

14. ed

15. a

16. state

17. -

18. of

19. -

20. the

21. -

22. art

23. demo

24. ,

25. but

26. user

27. s

28. did

29. n’t

30. read

31. the

32. FAQ

33. —

34. so

35. they

36. ask

37. ed

38. ,

39. “

40. Why

41. ’s

42. it

43. broken

44. ?

45. ”

46. Yesterday

47. ,

48. we

49. re-tested

50. v2.0

51. ,

52. fix

53. ed

54. 3

55. bug

56. s

57. ,

58. and

59. email

60. ed

61. support@example.com

62. .

2. Compare with a Tool

Run the paragraph through an NLP tool that supports your language (e.g., NLTK**,** spaCy, or any open-source tokenizer if available).

- Compare tool output vs. your manual tokens.

What will differ (typical differences you'll see)

1. **Clitics / contractions**

- **Manual (you split clitics):**
    - `can't → can, n't`
    - `it's → it, 's`
    - `didn't → did, n't`
    - `Why's → Why, 's`
    - `Kishore's → Kishore, 's`

- **Tool (NLTK word_tokenize usually keeps them as one token or splits differently):**
    - Often returns `can’t` as one token (or `ca`, `n't` depending on apostrophe type)
    - Often returns `it’s` as one token (or `it`, `'s`)
- **Why:** tokenizers make design choices: some preserve contractions as one token, others split to match English morphology for better modeling.

2. Hyphenated words

- **Manual:** you split `state-of-the-art` into `state - of - the - art`
- **Tool:** often keeps `state-of-the-art` as **one token**
- **Why:** many tokenizers treat hyphenated compounds as single lexical units unless configured otherwise.

3. Em dash / punctuation attachment

- **Manual:** `FAQ—so → FAQ, —, so`
- **Tool:** often returns `FAQ—so` as one token, or `FAQ`, `—`, `so`
- **Why:** em dashes aren’t always handled consistently; it depends on whether the tokenizer normalizes Unicode punctuation.

4. Quotes

- **Manual:** you separated curly quotes: `“` and `”` as their own tokens
- **Tool:** may normalize quotes, or keep them attached to nearby words
- **Why:** tokenizers differ in whether they output quote marks as separate tokens and whether they normalize Unicode quotes.

5. Email + period

- **Manual:** `support@example.com .` (period separated)
- **Tool:** often returns `support@example.com` and `.` separately (good), but some tokenizers may keep the trailing period attached
- **Why:** email/URL heuristics vary; some tools have special rules, others don’t.

6. **Version numbers**

- **Manual:** `v2.0` kept as one token
- **Tool:** might split into `v2.0` or `v2`, `.`, `0`
- **Why:** tokenizers differ on dotted numeric patterns; some treat them like abbreviations, some like punctuation + numbers.

- **Which tokens differ? Why?**

Tool tokenization differs from my manual tokens mainly on **contractions/possessives**, **hyphenated compounds**, and **Unicode punctuation** (em dashes and curly quotes). My manual tokenization intentionally split clitics (e.g., `didn't → did + n't`) and suffix-like endings (`shipped → ship + ed`) to expose morphology, while most general-purpose tokenizers keep many of these as single tokens to preserve surface form. Tools also apply different heuristics for punctuation: some always split punctuation into separate tokens, while others keep punctuation attached in cases like `FAQ—so` or normalize quotes. Finally, special strings like emails and version numbers may be handled differently depending on whether the tokenizer has explicit rules for them. These differences happen because tokenizers are optimized for different downstream tasks (parsing, search, language modeling, etc.), so "best" tokenization depends on the goal.

3. Multiword Expressions (MWEs)

Identify at least 3 multiword expressions (MWEs) in your language. Example:

- Place names, idioms, or common fixed phrases.
- Explain why they should be treated as single tokens.

Here are **3+ MWEs in English** (you can swap in your own language if you want). Each is a phrase that behaves like a **single unit of meaning**, so treating it as **one token** helps models avoid misinterpreting the parts.

1) New York City

- **Type:** place name (proper noun)
- **Why single token:** The meaning is one specific location; splitting into "New / York / City" can confuse the model (e.g., "new" as adjective, "city" as generic). As one token, it stays a stable entity.

2) state of the art

- **Type:** fixed phrase / technical idiom
- **Why single token:** It means "most advanced," not literally "art" or "state." If split, the phrase's meaning can get diluted and the model may treat it as a normal sequence of words instead of a single concept.

3) as soon as possible

- **Type:** fixed phrase (common formula)
- **Why single token:** People use it as one intent: "ASAP." If tokenized separately, the model might miss the fact that it's a single urgency signal.

4) kick the bucket

- **Type:** idiom
- **Why single token:** Means "to die," not literally kicking anything. Treating it as one token avoids incorrect literal interpretation.

General rule: MWEs should be single tokens when (1) the phrase has a meaning that isn't the sum of its parts, or (2) it names a specific entity, or (3) it's a fixed formula people use as one semantic unit.

4. Reflection (5–6 sentences)

- What was the hardest part of tokenization in your language?
- How does it compare with tokenization in English?
- Do punctuation, morphology, and MWEs make tokenization more difficult?

The hardest part of tokenization in my language was deciding what counts as a "word" when punctuation and attachments are involved, especially contractions/possessives (like *don't*, *it's*, *Kishore's*) and hyphenated compounds (like *state-of-the-art*). Compared with English, tokenization becomes harder whenever the language uses richer morphology or clitics, because a single surface word can contain multiple meaningful pieces that you may want to split for modeling. English is relatively easier because whitespace usually separates words, but even English becomes tricky with quotes, em dashes, abbreviations, emails/URLs, and version numbers. Yes—punctuation makes tokenization difficult because it can attach to words in many ways and Unicode symbols (curly quotes, en/em dashes) behave inconsistently across tools. Morphology makes it harder because you must choose between keeping words whole or splitting suffixes/prefixes depending on the task. MWEs add another layer because phrases like place names or idioms should often be treated as a single unit of meaning, even though they contain multiple words.